

FLIGHT DATA MANAGER

DANIELE TAMBONE



Contents

1	Introduction	1
1.1	Project Objectives	1
1.2	Methodological Approach	1
2	System Architecture	2
2.1	Architectural Diagram	2
2.2	Networking and Isolation Strategy	3
2.3	Communication Choices	3
3	User Manager Microservice	4
3.1	Application Core and Domain	4
3.2	Ports	4
3.3	Adapters	4
3.4	Repository	5
3.5	Middleware and Context Handling	5
3.6	“At-Most-Once” Implementation (Idempotency)	6
4	Data Collector Microservice	7
4.1	Domain and Core Application	7
4.2	Ports	7
4.3	Adapters	8
4.4	Repository	8
4.5	Worker	9
5	Kafka	9
5.1	Introduction and Role in Architecture	9
5.2	Data Flow and Order Guarantee	10
5.3	Implementation Details: Producer	10
5.4	Implementation Details: Consumer	11
5.5	Resilience Testing and Single-Node Limitations	11
	References	12

1 Introduction

This document illustrates the design choices and implementation details of the *Flight Data Manager* project, a distributed microservices-based system developed for the course in Distributed Systems and Big Data (A.Y. 2025-2026).

The goal of the project is to build a scalable and resilient platform for user management and real-time monitoring of air traffic, based on data provided by the public API of the OpenSky Network [1].

1.1 Project Objectives

The system was designed to meet strict functional and non-functional requirements typical of modern cloud-native architectures:

- **Microservices Decomposition:** The application domain was divided into two autonomous services, *User Manager* and *Data Collector*.
- **Resilience and Robustness:** Implementation of failure-handling patterns, including *At-Most-Once* messaging policies, Retry, and Circuit Breaker.
- **Security:** Secure credential management, network isolation (Micro-segmentation), and cross-service authorization checks.
- **Hybrid Storage:** Use of different databases, choosing *Redis* for pure speed (key-value data and idempotency handling) and *Neo4j* for modeling complex relational data (graphs).

1.2 Methodological Approach

Software development was guided by the principles of **Hexagonal Architecture** (or *Ports and Adapters*). This choice allowed us to isolate the *Business Logic* (the Core) from infrastructural details such as databases, HTTP frameworks, or communication protocols.

Such decoupling enables code testability and system maintainability, allowing adapters to be modified without impacting domain rules. This modularity was extended to the infrastructure level: the entire system was containerized using **Docker** to ensure maximum portability.

2 System Architecture

2.1 Architectural Diagram

The system is composed of five main containers orchestrated through Docker Compose. Figure 1 illustrates the components and their interactions.

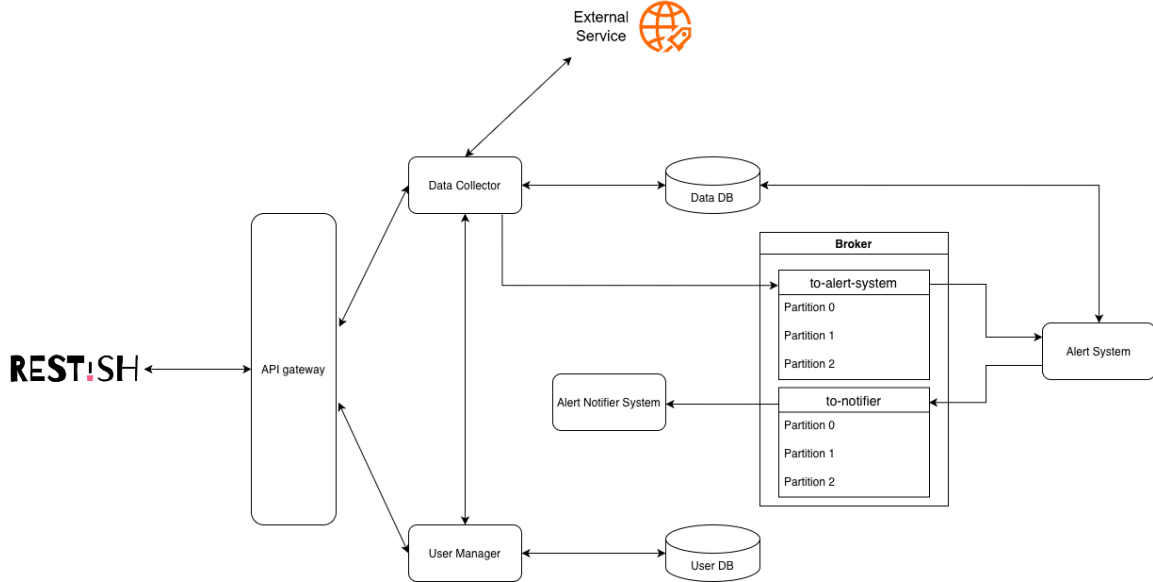


Figure 1: Architectural diagram of the Flight Data Manager system.

- **Restish:** A CLI (Command Line Interface) client used during development and testing to simulate frontend/client behavior. It serves as the interface used to test the REST APIs exposed by the Gateway.
- **External Service:** Public API of the OpenSky Network.
- **API Gateway:** Implemented with *Nginx*, it acts as the single entry point for external traffic, handling SSL termination and routing HTTPS requests to the appropriate microservices (*User Manager* or *Data Collector*).
- **User Manager:** Microservice responsible for identity management. It implements the gRPC service for credential verification and the *At-Most-Once* policy for user registration.
- **Data Collector:** It manages user interests, runs the asynchronous worker to fetch data from the External Service, and acts as a Producer pushing flight data events to the Broker.
- **Broker:** Implemented with *Kafka* (Kraft mode). It manages asynchronous communication via partitioned topics

- **Alert System:** It consumes data from the `to-alert-system` topic, evaluates alert conditions against user interests and produces notification events to the `to-notifier` topic.
- **Alert Notifier System:** The final consumer in the pipeline. It listens to the `to-notifier` topic and handles the delivery of notifications to the end users (e.g., via email adapter).
- **User DB:** Database implemented with *Redis*, used for storing user profiles and for atomic management of idempotency locks.
- **Data DB:** Database implemented with *Neo4j*, chosen for its native modeling of relationships between users, interests, and flights.

2.2 Networking and Isolation Strategy

One of the key design choices was the adoption of **Micro-segmentation** at the Docker network level to ensure security according to the "Least Privilege" principle. Three distinct networks were defined:

- **gateway-net:** Connects the API Gateway to the two microservices. Allows inbound HTTP traffic.
- **service-net:** A private network dedicated to *inter-service* (gRPC and Kafka) communication.
- **user-net** and **data-net:** Isolated backend networks that connect each microservice exclusively to its own database.

This configuration physically prevents, for example, the *Data Collector* from accessing the users' *Redis* database, thereby reducing the attack surface.

2.3 Communication Choices

2.3.1 External: REST (Huma)

For interaction with clients (end users), REST APIs were exposed. The choice of the **Huma** framework ensured strict input validation (via OpenAPI) and automatic documentation generation.

2.3.2 Internal: gRPC

For synchronous communication between microservices (specifically, when the *Data Collector* needs to verify a user's identity with the *User Manager*), **gRPC** was chosen instead of REST. The motivations for this design choice include:

1. **Performance:** The use of *Protocol Buffers* (binary) reduces latency and data volume compared to JSON, which is essential for frequent internal calls.

2. **Strong Typing:** The contract defined in the `.proto` file ensures that client and server are always aligned on data types, preventing integration errors.
3. **Rich Error Handling:** Adoption of the `google.rpc.Status` standard enables structured and detailed error reporting between services.

3 User Manager Microservice

The **User Manager** is the component responsible for handling the user lifecycle and credential security. It acts as the central identity *Authority* within the distributed system. It is designed to be **stateless**, delegating state persistence to Redis.

3.1 Application Core and Domain

The core of the microservice is defined by the **Domain**, which specifies the **User** entity and its invariant validation rules. To guarantee data consistency, the validation logic is encapsulated directly within the domain *Factory Function* (`NewUser`). This ensures that a user object can never be instantiated in an invalid state (e.g., malformed email or missing required fields), protecting the system from corrupted data.

The **Application** layer orchestrates the use cases (Registration, Deletion), coordinating data flow between input and output ports without depending on specific technologies.

3.2 Ports

The ports define the interaction contracts of the Core:

- **UserAPI (Driver):** Defines the operations available to clients, including methods for idempotency management.
- **UserRepository (Driven):** Abstracts persistence operations, allowing the Core to save and retrieve users while remaining unaware of the underlying Redis implementation.

3.3 Adapters

Adapters connect the Core to the outside world:

- **REST Adapter (Huma):** Handles public HTTP requests, validating inputs via OpenAPI schemas and mapping domain errors (e.g., `ErrUserNotFound`) to appropriate HTTP status codes (e.g., 404).
- **gRPC Adapter:** Exposes an interface used by the *Data Collector* to check whether a user exists (`CheckUserExistence`) and to verify credentials (`VerifyCredentials`) with minimal latency.

- **Redis Repository:** Implements data persistence by leveraging Redis' low latency to store user profiles serialized as JSON. It also manages critical concurrency through atomic commands (**SETNX**), which are essential for the idempotency locking mechanism.

3.4 Repository

For persistence, **Redis** was chosen, an in-memory key-value database. This choice is motivated by the need for:

- **Low Latency:** Identity verification operations are critical and frequent.
- **Simplicity:** The data model is denormalized. Each user is serialized as JSON and stored under a composite key: `user:<email>`.
- **Atomicity:** Commands like **SETNX** (Set if Not Exists) are essential for preventing race conditions during concurrent registrations.

3.5 Middleware and Context Handling

Adopting a microservice architecture with an API Gateway (Nginx) introduces a technical challenge related to identity and network origin: *IP address masking*. Since all incoming traffic passes through the Gateway, the *User Manager* technically sees every request as originating from the internal Nginx container IP (e.g., `172.18.0.x`), losing the actual client IP information.

This is a critical issue for the *Scoped Idempotency* mechanism, which relies on caller IP differentiation for security.

3.5.1 IP Extraction Middleware

To solve this issue, a custom **HTTP Middleware** (`internal/middleware`) was implemented to intercept each request before it reaches the application layer (Huma).

The middleware applies a priority-based extraction strategy to determine the real originator of the request:

1. **X-Forwarded-For header:** The de-facto standard for reverse proxies. If present, the middleware extracts the first IP in the list (the original client), trusting the proxy chain configured in Nginx.
2. **X-Real-IP header:** Used as a fallback when configured by specific load balancers.
3. **RemoteAddr:** Used as a last resort (direct connection) when the service is accessed without passing through the Gateway, such as during local testing or debugging.

3.5.2 Context Injection

Once extracted, the real IP is not passed explicitly as a parameter to controller methods; instead, it is injected into the request's `context.Context`. This design choice, idiomatic in Go, allows the “contextual” information (who is making the request) to propagate transparently through the application layers, making it accessible to the Handler for idempotency hash computation without polluting business-logic function signatures.

3.6 “At-Most-Once” Implementation (Idempotency)

To satisfy the *At-Most-Once* messaging requirement and robustly handle automatic client retries (or network errors during ACK delivery), a **Scoped Idempotency** mechanism was implemented.

3.6.1 The Problem

In a distributed system, if a client sends a registration request and does not receive a response (time-out/network error), it will attempt to send it again. Without idempotency, this could result in:

- Creation of duplicate users.
- Spurious errors (“User already exists”) that confuse the user, who believes the registration failed even though it actually succeeded.

3.6.2 The Solution: Idempotency Key and Data Structure

The client must generate a unique ID for each operation attempt (`X-Request-ID` header). The server uses this ID to track the request state in Redis.

The Redis lock key is built by combining the caller identity and the request ID:

$$Key = \text{idempotency} : \text{SHA256}(IP_{client}) : \text{RequestID} \quad (1)$$

Including the client IP hash prevents accidental collisions among different users who might reuse the same ID.

The value associated with the key is not a simple flag but a JSON structure storing the full operation state:

- **Status:** Indicates the current state (`PROCESSING` or `COMPLETED`).
- **Response:** A serialized copy of the response body (JSON) sent to the client. This allows returning the exact same original response in case of retries.
- **Timestamp:** The time when the first request was received.

3.6.3 States

The registration process follows two states, managed atomically in Redis:

1. **PROCESSING:** As soon as a request with a new ID arrives, the system acquires an atomic *lock* (via `SETNX`) storing the `PROCESSING` state. If duplicate requests arrive while the lock is active, they are rejected (`429 Too Many Requests`) to avoid race conditions.
2. **COMPLETED:** Once processing completes, the Redis record is updated: the state becomes `COMPLETED` and the `Response` is stored. If the client retries with the same ID, the system immediately returns the stored snapshot, simulating an instant success without touching the user database.

In case of a blocking application error during registration (e.g., invalid password), the system performs a **ROLLBACK**, deleting the idempotency key. This restores the initial state, allowing the client to retry the operation with the same ID (e.g., after correcting the parameters).

4 Data Collector Microservice

The **Data Collector** represents the analytical engine of the platform, responsible for aggregating, storing, and retrieving flight data. Unlike the User Manager, which has a purely reactive nature, this microservice adopts a hybrid model that combines synchronous interaction with the user and background processing for database population.

4.1 Domain and Core Application

The core of the service is modeled around two main entities: **Interest** and **Flight**. The **Interest** entity encapsulates the user's intent to monitor a specific airport, including validation logic to ensure ICAO code compliance. The **Flight** entity, on the other hand, acts as a normalized data transfer object (DTO), abstracting away the complexity and heterogeneity of the raw data provided by the external provider OpenSky Network.

The Core orchestrates two distinct yet interconnected workflows. The first is the *user flow*, dedicated to handling real-time requests (interest registration, statistics queries), where each operation is subject to strict credential verification via gRPC. The second is the *worker flow*, a fully decoupled background process responsible for the entire data lifecycle, from acquisition to cleaning.

4.2 Ports

Ports define the boundaries of the system:

- **CollectorAPI (Driver):** Exposes business functionalities, including batch management of interests and statistical queries.
- **FlightRepository (Driven):** Abstracts persistence on the graph database, providing specific methods for relational queries (“The user follows airport X”).
- **ServiceClient (Driven):** Defines contracts for external dependencies (*User Manager* and *OpenSky*).

4.3 Adapters

Adapters connect the Core to the external world, handling both inputs (from users or time) and outputs (towards databases or third-party services):

- **REST Adapter (Huma):** As in the User Manager, it handles the public HTTP interface. In addition to input validation, it manages DTO-based data transformation: it converts internal Unix timestamps into human-readable formats (ISO 8601) and translates domain security errors (e.g., `ErrUserNotAuthorized`) into appropriate HTTP responses (e.g., 401).
- **gRPC Client Adapter:** Manages synchronous communication with the *User Manager*. It implements client-side resilience patterns such as **Retry** (for transient network errors) and **Circuit Breaker** (to prevent cascading failures), making the system robust even in case of User Manager unavailability.
- **Neo4j Repository:** Translates domain operations into *Cypher* queries. It leverages features such as `UNWIND` for batch insertion of interests and `MERGE` to ensure idempotency at the database level, preventing duplicates in the graph.
- **OpenSky Client:** Extracts data via the public OpenSky Network API.
- **Worker Adapter:** A *Driving* adapter acting as a temporal trigger. It decouples time management from business logic, periodically initiating data collection and cleaning processes.
- **Kafka Producer Adapter:** Implements the asynchronous communication channel towards the message broker. It is responsible for publishing events to the `to-alert-system` topic.

4.4 Repository

A *Domain-Specific* persistence strategy was adopted, selecting **Neo4j** as the storage technology. This choice is motivated by the intrinsic topology of the data, which naturally lends itself to a graph repre-

sentation: users are connected to airports through interest relationships (`INTERESTED_IN`), while flights connect departure and arrival airports (`DEPARTED`, `ARRIVED`).



Figure 2: Database schema.

Using a graph database provides significant performance advantages over traditional relational joins. Additionally, data integrity and consistency are ensured at the query level: the `MERGE` clause makes flight insertion an idempotent operation, preventing duplicates even when data import cycles are re-executed, while the use of `UNWIND` supports efficient batch processing for mass insertion of interests.

4.5 Worker

Data acquisition is handled by a cyclic *Background Worker* (with a configurable frequency, default 12 hours). To optimize resource usage and respect the rate-limiting constraints imposed by external APIs, a *Checkpointing* strategy has been implemented. The system stores, for each airport, the timestamp of the last update: at every cycle, the worker evaluates the “freshness” of the data and, if the update is recent, avoids redundant calls to OpenSky. Otherwise, it proceeds with *Incremental Fetching*, downloading only the missing time window.

5 Kafka

5.1 Introduction and Role in Architecture

In compliance with the project requirements, a Message Broker (Apache Kafka) has been introduced to decouple system components. The architecture evolves from synchronous communication to an **event-driven** one, improving resilience and scalability.

The integration follows the principles of **Hexagonal Architecture**:

- The **Producers** (Driven Adapters) publish events externally.
- The **Consumers** (Driver Adapters) react to incoming events.

The variable `KAFKA_NUM_PARTITIONS: 3` sets the default to 3 partitions for automatically created topics. This configuration prepares the system for horizontal scalability (Parallel Consumer), allowing multiple instances of the same service to process messages in parallel.

5.2 Data Flow and Order Guarantee

The system uses two main topics. Despite the presence of multiple partitions, temporal data integrity is guaranteed by the strict use of **Message Keys**.

5.2.1 Topic to-alert-system

- **Source:** `data-collector` → **Destination:** `alert-system`
- **Payload:** `AirportUpdateEvent` (Arrival/Departure flight).
- **Partition Key (Key):** `AirportCode`.

Using the airport code as a key ensures that all updates related to the same airport (e.g., "LIRF") are always routed to the same partition (thanks to consistent hashing). This guarantees that the `alert-system` receives chronological updates in the exact order of generation, avoiding race conditions on timestamps.

5.2.2 Topic to-notifier

- **Source:** `alert-system` → **Destination:** `alert-notifier-system`
- **Payload:** `Notification` (Email).
- **Partition Key (Key):** `UserEmail`.

By using the user's email as a key, we guarantee that all notifications directed to a specific person are handled sequentially, preventing the arrival of disordered alerts.

5.3 Implementation Details: Producer

Producers are configured for maximum reliability (`acks: "all", retries: 3`).

- **Asynchronous Delivery:** A dedicated goroutine (`deliveryReportHandler`) monitors the sending outcome without blocking the main thread.

- **Throughput Optimization:** `linger.ms: 10` introduces a slight artificial delay (10ms) before sending a batch. This forces the producer to wait for more messages to arrive, resulting in larger batches and fewer network requests.
- **Strict Ordering:** The `max.in.flight.requests.per.connection: 1` configuration is fundamental: in case of a retry due to a network error, it prevents a subsequent message from being written before a previous one, maintaining order even in the face of transient failures.

5.4 Implementation Details: Consumer

Consumers adopt an "At-Least-Once" approach with manual commit management.

- **Polling & Logic:** The consumer performs continuous polling. Upon receipt, the message is deserialized and passed to the business logic (Core).
- **Manual Commit:** The `CommitMessage` method is invoked **only after** the business logic (`CheckThreshold` or `NotifyUser`) has successfully completed. If the service crashes during processing, the message is not committed and will be re-fetched upon restart.
- **Offset Reset:** `auto.offset.reset: "earliest"` ensures that no message is lost if the consumer starts for the first time or if the current offset is invalid.

5.5 Resilience Testing and Single-Node Limitations

During stress testing phases—specifically simulating a Broker outage and subsequent restart during an active ingestion cycle—it was observed that messages buffered in the Producer’s internal queue were sometimes rejected upon reconnection with the error: `Unknown topic or partition`.

This behavior is intrinsic to the **Single-Node** configuration deployed in an ephemeral containerized environment. Upon restart, the solitary broker requires a distinct warm-up period to initialize metadata and recover logs. In the absence of an active cluster, there are no peer nodes to maintain topic state or partition leadership during the outage, leading to a temporary unavailability of metadata.

It is important to note that in a real-world production environment, this issue is addressed at the **infrastructure level** rather than the application level guaranteed by deploying a **Kafka Cluster** with a **Replication Factor** of $N > 1$.

References

- [1] M. Schäfer, M. Strohmeier, V. Lenders, I. Martinovic, and M. Wilhelm, “Bringing Up OpenSky: A Large-scale ADS-B Sensor Network for Research,” in *Proceedings of the 13th IEEE/ACM International Symposium on Information Processing in Sensor Networks (IPSN)*, IEEE/ACM, Apr. 2014, pp. 83–94.